

Technical SEO Field Guide

A practical implementation reference for crawling, indexing, canonicalization, redirects, JavaScript rendering, performance, structured data, internal linking, and Search Console diagnostics.

How to use it

Treat each section as an audit procedure. For every important URL, establish what should happen, inspect the actual production response and HTML, then verify the result with the appropriate search or performance tool.

Technical SEO objective

Make the intended URLs discoverable, crawlable, renderable, indexable, understandable, and measurable. Technical correctness does not guarantee search visibility; relevance, content quality, search intent, competition, and other systems also affect search results.

This edition reflects documentation available on September 24, 2026. Re-check source documentation when implementing major changes because search systems and supported features can change.

1. Crawlability & robots.txt

Crawling is the process of requesting URLs and resources. robots.txt is a crawler-access control mechanism. It is not an index-removal mechanism.

Correct root location

`https://example.com/robots.txt`

Rules apply to the host, protocol, and port where the robots.txt file is served. The Sitemap directive should use an absolute URL.

```
User-agent: *  
Disallow: /admin/  
Disallow: /private/
```

Sitemap: `https://example.com/sitemap.xml`

Audit

- Fetch /robots.txt directly and verify HTTP 200.
- Confirm important public HTML paths are not disallowed.
- Confirm required CSS, JavaScript, and image resources are not unintentionally blocked.
- Confirm staging, preview, test, and development paths are not exposed unintentionally.
- Confirm the sitemap URL is absolute and points to the production sitemap.
- Test representative production URLs in Search Console URL Inspection.

Critical distinction

If a URL must be evaluated for a noindex directive, the crawler must be able to access the URL. Blocking it in robots.txt can prevent the crawler from seeing the noindex instruction.

Common failure

Disallowing an entire asset directory can prevent rendering resources from being fetched. Conversely, allowing everything while accidentally exposing administrative or private paths creates a crawl-control problem. Audit rules against the actual site architecture.

2. XML Sitemaps & URL Discovery

A sitemap helps search engines discover URLs. It should represent the site's intended indexable URL set, not every URL the server can technically serve.

Production sitemap rules

- Use absolute HTTPS URLs.
- List the preferred canonical URL for each page.
- Exclude redirects, 404/410 URLs, noindex pages, and obvious duplicates.
- Keep XML valid and UTF-8 encoded.
- Update entries when important URLs materially change.
- Submit the sitemap in Search Console.
- Reference the sitemap from robots.txt.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://example.com/</loc>
    <lastmod>2026-09-24</lastmod>
  </url>
</urlset>
```

Three-way URL reconciliation

Compare: **sitemap URLs** ↔ **internal-link URLs** ↔ **Search Console indexed/canonical URLs**. A URL in the sitemap that redirects or has noindex is a contradiction. An important page absent from both the sitemap and internal links deserves investigation.

Do not confuse discovery with indexing

Submitting a URL in a sitemap does not force indexing. The page still needs to be accessible, eligible for indexing, and processed by Google's systems.

3. HTTP Status, Redirects & Errors

The HTTP response is foundational. Test the final production response, not merely what a browser appears to display.

Status	Use	Audit rule
200	Successful resource	Normal response for an indexable page.
301	Permanent redirect	Use for a permanent URL move; target the final destination.
302 / 307	Temporary redirect	Use only when the move is genuinely temporary.
404	Not found	Use when no resource exists and no replacement is appropriate.
410	Gone	Explicitly indicates permanent removal.
5xx	Server failure	Investigate promptly for important URLs.

Redirect chain

Avoid:

A -> 301 -> B -> 301 -> C -> 200

Prefer:

A -> 301 -> C -> 200

Redirect audit

- Every retired important URL has a deliberate replacement or an intentional 404/410.
- Permanent moves use permanent redirects.
- Redirects resolve directly to the final URL where practical.
- No redirect loops.
- No long HTTP/HTTPS/www/non-www chains.
- Internal links point directly to final URLs.
- Do not redirect unrelated obsolete URLs to the homepage merely to avoid 404s.

4. Canonicalization

Canonicalization selects a representative URL among duplicate or substantially similar URLs. `rel=canonical` is a signal, not an absolute command; Google can choose a different canonical.

Self-canonical example

```
<link rel="canonical"
      href="https://example.com/services/seo.html">
```

Canonical consistency model

For an ordinary indexable page, align these signals: **final URL** → **internal links** → **canonical tag** → **sitemap** → **preferred host/protocol**.

Failure patterns

- Canonical points to HTTP while production is HTTPS.
- Canonical points to the old domain.
- Canonical points to the homepage on unrelated pages.
- Sitemap URL differs from the canonical URL.
- Redirected URLs appear as canonical URLs.
- Parameter or filter URLs create unintended duplicate pages.
- Two URL variants serve materially identical content but have no deliberate canonical strategy.

Verification

Inspect the original HTML source, then use Search Console URL Inspection to compare the declared canonical with Google's selected canonical. A mismatch is a diagnostic signal—not automatically a defect—but it should be understood.

5. Indexability & Robots Meta

Indexability asks whether a crawlable URL should be eligible for indexing. Most ordinary public pages do not need an explicit index directive.

Examples

```
<!-- Default-style explicit directive -->
<meta name="robots" content="index, follow">

<!-- Prevent indexing -->
<meta name="robots" content="noindex, follow">
```

Indexability checklist

- Important pages return an appropriate successful response.
- Important pages do not contain an unintended noindex directive.
- Important pages are not blocked from crawling.
- Canonical points to the intended representative URL.
- Important pages are internally linked.
- Important pages are included in the sitemap when appropriate.
- Private/account/admin pages are intentionally controlled.
- Search, sort, filter, cart, and utility URLs have deliberate indexation behavior.

Soft 404

A URL can return HTTP 200 while presenting an empty, missing, or error-like experience. For important URLs, verify both the HTTP status and the actual content returned.

6. JavaScript SEO & Rendering

JavaScript can create content, navigation, metadata, and structured data. The implementation must remain understandable and testable after rendering.

Primary links should be real links

Good:

```
<a href="/services/seo.html">Search Engine Optimization</a>
```

Risky as primary navigation:

```
<div onclick="location.href='/services/seo.html'">  
  Search Engine Optimization  
</div>
```

Rendering audit

- Important content is available in the rendered page.
- Primary navigation uses real href links.
- Important URLs do not depend on fragile click handlers for discovery.
- Canonical URLs remain consistent before and after rendering.
- No accidental noindex is injected by JavaScript.
- Structured data remains valid after rendering.
- Lazy loading does not hide essential content from normal page processing.
- Core page content remains usable if nonessential JavaScript fails.

Canonical caution

Google's current documentation notes that canonicalization occurs before and after rendering. Avoid changing the canonical unexpectedly during JavaScript execution; keep the intended URL unambiguous.

7. Performance & Core Web Vitals

Measure representative pages using both field data and controlled testing. The current Core Web Vitals thresholds are:

Metric	Good	Technical focus
LCP	≤ 2.5 s	Largest main-content loading experience.
INP	≤ 200 ms	Responsiveness to interaction.
CLS	≤ 0.1	Unexpected layout movement.

Common causes

- Oversized hero images or inefficient formats.
- Slow server response.
- Large JavaScript bundles or long main-thread tasks.
- Excessive third-party scripts.
- Render-blocking CSS or excessive CSS.
- Fonts that delay rendering or cause layout changes.
- Images without dimensions that cause layout shifts.
- Unnecessary API/network requests.

Diagnostic order

Identify the largest bottleneck first. Separate server response, network transfer, rendering/main-thread work, and layout stability. Avoid treating minification as a universal solution when the dominant issue is an oversized image, third-party script, server response, or JavaScript execution.

8. Structured Data

Structured data is machine-readable information about page entities. It can help Google understand content and can make a page eligible for supported rich-result features; markup alone does not guarantee a rich result.

JSON-LD example

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "Example Company",
  "url": "https://example.com/",
  "logo": "https://example.com/logo.png"
}
</script>
```

Implementation audit

- Choose a schema type that accurately describes the page or entity.
- Use supported properties and required fields for the intended feature.
- Keep markup consistent with the actual page content.
- Validate syntax and required properties.
- Test the live production URL after deployment.
- Monitor relevant Search Console enhancement reports.
- Remove or update stale markup when the underlying page changes.

Rule

Do not add structured data simply because a schema type exists. Mark up what the page actually represents, and validate the deployed implementation.

9. Internal Linking & Architecture

Internal links help users navigate and help crawlers discover and contextualize URLs. The objective is not maximum link count; it is a coherent, crawlable information architecture.

Example

```
<a href="/services/local-seo.html">  
  Local SEO Services  
</a>
```

Architecture audit

- Every important page is reachable through crawlable internal links.
- Important pages are not orphaned.
- Anchor text describes the destination when practical.
- Service pages connect to relevant solutions, guides, and supporting content.
- Internal links use final URLs instead of redirecting URLs.
- Broken internal links are repaired.
- Navigation reflects the site's actual content hierarchy.
- URL depth is reasonable for important pages.

Orphan-page test

Compare URLs discovered from internal links against URLs listed in the sitemap. Pages that appear only in a sitemap and have no meaningful internal path deserve review.

10. Search Console & Deployment QA

Search Console is a diagnostic system. Use it after launches, template changes, URL migrations, structured-data changes, and significant technical deployments.

URL Inspection workflow

Step	Action
1	Enter the exact production URL.
2	Check indexing status.
3	Compare declared and selected canonical URLs.
4	Inspect crawl/access information.
5	Use live testing when verifying the current deployment.
6	Request indexing only after the technical issue is fixed.

Deployment acceptance checklist

- Production URL returns the intended HTTP status.
- HTTPS and preferred host are correct.
- robots.txt is accessible and not over-blocking.
- Sitemap contains intended indexable URLs.
- Canonical tags match production URLs.
- No accidental noindex exists.
- Primary navigation uses crawlable links.
- Important content is present in rendered output.
- Structured data validates where implemented.
- Performance regression has been reviewed.
- Representative URLs have been inspected in Search Console.

11. Troubleshooting Matrix

Symptom	Check first
Page not indexed	HTTP status → noindex → robots.txt → canonical → internal links → sitemap → URL Inspection.
Google selected another canonical	Canonical tag → redirects → sitemap → internal links → duplicate URL variants → URL Inspection.
Important page not discovered	Internal links → sitemap → robots.txt → navigation → orphan status.
Traffic drops after deployment	Canonical → noindex → robots → redirects → template content → status codes → tracking.
Rich result disappears	Structured-data validity → required properties → visible content → Search Console report.
Page becomes slow	LCP resource → server response → JavaScript/main thread → images → third parties → layout shifts.
Old URL persists	301 mapping → sitemap → internal links → canonical → old URL response.
Duplicate URLs appear	Parameters → slash/case variants → host/protocol → canonical → redirects.

Authoritative references

Google Search Central — robots.txt specification

<https://developers.google.com/crawling/docs/robots-txt/robots-txt-spec>

Google Search Central — canonicalization

<https://developers.google.com/search/docs/crawling-indexing/canonicalization>

Google Search Central — canonicalization troubleshooting

<https://developers.google.com/search/docs/crawling-indexing/canonicalization-troubleshooting>

Google Search Central — structured data

<https://developers.google.com/search/docs/appearance/structured-data>

Google Search Central — Search appearance

<https://developers.google.com/search/docs/appearance>

Google Search Central — site moves and URL changes

<https://developers.google.com/search/docs/crawling-indexing/site-move-with-url-changes>

web.dev — Core Web Vitals

<https://web.dev/articles/vitals>

Revision: September 24, 2026